Case 02 · Spec: Private Exposure Registry

Chapter 2 (Specification) of the Magshimim project-book template: features, technologies, flow diagram and database structure. The document is written so that another developer could build the system from it. Each feature is described at the level of "code in words": what it does, how it works step by step, which functions and data structures, which industry tools complement it, and what its inputs and outputs are.

System overview

What the system does, in one sentence

Private Exposure Registry lets supervised financial institutions report their rate and leverage exposures in split, encrypted form, so that three servers at three supervisory authorities jointly compute an aggregate picture of the market, without any of them, or anyone else, seeing a single participant's data.

What changed since the POC and the threat model

- **The POC proved the core:** splitting into shares, aggregating, and opening only the sum, in 0.11 seconds for 500 participants. But it assumed "honest but curious" servers and honest participants.
- The threat model found six holes, and this is the document where it changes the design more than anywhere else: a Sybil attack (fake funds that bypass the threshold), a server tampering with shares, a participant who denies, a differencing attack, a server going down, and a participant who lies.
- **We checked what the industry has already built:** there is a standard that does exactly this core. Prio3, part of the VDAF draft (Verifiable Distributed Aggregation Functions) from the IETF's cryptography research group, which reached version 21 in August 2026. Prio was used in practice for private statistics in COVID exposure-notification apps. So we will not build our own protocol: we build on the standard and add on top what it does not provide.

A key design decision: how many servers must be honest?

The threat model left a decision open: all three servers needed (more secret, but one server going down stops everything), or any two sufficient (more available, but two colluding servers break secrecy). The decision: all three shares are needed. The reason: in Prio3, secrecy holds as long as at least one of the servers is honest. To expose a fund's data, all three authorities would have to collude in secret, a far more remote scenario than collusion between two. The availability cost is paid in operations: each server is a cluster with failover, and encrypted reports are held until the server returns (feature 8). This is a conscious trade-off: we prefer a delayed report to a leak.

Feature list

- Feature 1, Participant Onboarding & Credentials
- Feature 2, Reporting Template & Local Encoding
- Feature 3, Commitments & Receipts
- Feature 4, Prio3 Sharding & Validity Proofs
- Feature 5, Three-Aggregator Protocol

- Feature 6, Verifiable Aggregate
- Feature 7, Query Catalog, Threshold & Privacy Budget
- Feature 8, Transport, HPKE & Operations
- Feature 9, Audit & Transparency Log
- Feature 10, Regulator Reports & Public Release

Feature 1: Participant Onboarding & Credentials

What does the feature do?

Decides who may report. Only a genuine supervised entity, identified by a regulator, receives permission to participate. This is the answer to the Sybil attack in the threat model: if a fake fund cannot be registered, a cell cannot be "inflated" to pass the minimum threshold.

How does it work?

Identification by LEI

Every financial entity active in the markets holds an LEI (Legal Entity Identifier): a 20-character identifier under ISO 17442, issued and validated by GLEIF and its partners. The regulator checks that the applicant's LEI is active in GLEIF's public database and that it appears on the list of entities it supervises. One entity, one LEI: ten funds cannot be registered from the same entity.

Anonymous participation credentials

There is a subtle problem here: if the servers know which LEI sent each report, they know who participates, and the threat model flagged participation itself as sensitive. The solution: blind tokens. Once per reporting period, the regulator issues each verified participant one signed token, using a blind signature: the regulator signs without seeing the token itself. When the participant sends a report, it attaches the token. The servers check that the regulator's signature is valid and that the token has not been used, but cannot link it to an LEI. So: only real entities report, each entity reports once, and no server knows who.

Counting real participants

The minimum threshold in feature 7 counts valid tokens in a cell, not reports. Since each entity receives one token, "five participants" really are five different entities.

Complementary industry technologies and infrastructure

- **GLEIF API:** the public interface to the global LEI database. Contribution: identity verified against an authoritative global source, not an internal list.
- **Privacy Pass (RFC 9576 and RFC 9578):** an IETF standard for blind tokens, used in practice by Cloudflare and Apple to prove "I am a real user" without revealing "which user". Contribution: the mechanism we described is exactly Privacy Pass, with RSA Blind Signatures (RFC 9474). No invented cryptography.
- **smallstep step-ca:** an open certificate authority for managing X.509 certificates. Contribution: the regulators themselves, and the servers, identify each other with certificates.

Data structure

```
Participant {            # stored at the regulator only
```

```
  lei: str               # ISO 17442, 20 chars
  regulator_id: str
  sector: str
  verified_at: datetime
  status: Literal["active", "suspended"]
}

ParticipationToken {     # what the aggregators see
  period_id: str         # e.g. "2026-Q4"
  token: bytes           # unblinded token
  signature: bytes       # regulator RSA blind signature
}
```

Input and output

Input: a joining request with an LEI. Output: a Participant record at the regulator, and one anonymous participation token per period.

Key libraries and functions

Operation	Library / API	Usage
LEI verification	GLEIF API	GET /api/v1/lei-records/{lei}
Blind signature	Privacy Pass, RFC 9474	blind(), blind_sign(), finalize()
Server certificates	step-ca	X.509, mTLS

Feature 2: Reporting Template & Local Encoding

What does the feature do?

A client application that runs at the participant, takes its loan book, and computes the uniform reporting vector from it. The raw data never leave the participant's machine. This is exactly the design decision the POC validated: bucketing happens locally, so the servers only need to add.

How does it work?

The vector's structure

The vector consists of fixed cells, defined in the period's template version:

- **Exposure by rate bucket and sector:** four rate buckets (below 3%, 3 to 4, 4 to 5, above 5), times seven sectors. Each cell is total dollar exposure, rounded to the million.
- **Leverage bucket:** five buckets (below 2, 2 to 3, 3 to 4, 4 to 5, above 5). A "one-hot" vector: one in the participant's bucket, zero elsewhere.
- **Floating versus fixed-rate exposure:** two cells.
- **A count cell:** always 1, to count participants in the same aggregation.

Clipping

Every exposure cell is clipped to a cap (say 400 million per cell). The reason is twofold: it bounds how much one participant can influence the sum (needed for the noise calculation in feature 7), and it bounds the damage a participant trying to disrupt can do. The participant is warned when a cell is clipped, and how much of its value was clipped is kept locally for audit.

Encoding into a finite field

The cryptography works on integers in a finite field (modulo a prime). Each value is converted to a non-negative integer, and each value is limited to the number of bits the template sets (for example 9 bits per exposure cell, because 400 is less than 512). The bit limit is what later makes it possible to check that the value is in range (feature 4).

What the participant sees

Before sending, the application shows the participant its vector in a readable table ("this is what you are about to report"), and the participant confirms. Nothing that is not in the table is sent.

Complementary industry technologies and infrastructure

- **Rust with WebAssembly:** the client application is written in Rust and compiled both as a desktop application and to WebAssembly in the browser. Contribution: the same cryptographic library (feature 4) everywhere, and memory safety.
- **Apache Arrow:** an in-memory columnar format. Contribution: reading a loan book of hundreds of thousands of rows from the participant's Parquet or CSV files, and bucketing it quickly.
- **JSON Schema for the template:** every template version is published as a schema. Contribution: client, servers and regulator agree exactly on the vector's structure, and every template change is a new version with an identifier.

Data structure

```
Template {
  template_id: str            # "exp-v3"
  rate_buckets: list[float]   # [3.0, 4.0, 5.0]
  sectors: list[str]          # 7 sectors
  lev_buckets: list[float]    # [2, 3, 4, 5]
  clip_musd: int              # 400
  bits_per_cell: int          # 9
}

LocalReport {                 # never leaves the participant's machine in clear
  template_id: str
  cells: list[int]            # length L, each in [0, 2^bits)
  clipped_cells: list[int]    # indices that were clipped (kept locally)
}
```

Input and output

Input: the participant's loan book (a local file) and the period template. Output: a fixed-length encoded vector, ready for the commitment and splitting stage.

Key libraries and functions

Operation	Library	Usage
Read loan book	Apache Arrow (arrow-rs)	ParquetRecordBatchReader
Bucketing	Rust std	HashMap<(rate_bucket, sector), u64>
Browser execution	wasm-bindgen	exporting functions to JavaScript

Feature 3: Commitments & Receipts

What does the feature do?

Before the participant splits its data, it "locks" them in a cryptographic commitment: a value that proves what it reported without revealing it. This answers the "participant who denies" threat: later, in an audit, the participant can be asked to open the commitment, and it cannot claim to have reported something else. And an important bonus: the commitments make it possible to check that the servers computed correctly (feature 6).

How does it work?

A Pedersen commitment

For each cell in the vector, the participant computes a Pedersen commitment: a point on an elliptic curve equal to x times a point G plus r times a point H, where x is the cell's value and r a secret random number. Three properties make it suitable:

- **Hiding:** without r, the commitment reveals nothing about x.
- **Binding:** it is impossible to find another x and another r that give the same commitment.
- **And the most important property here, homomorphic:** adding commitments equals a commitment to the sum. Adding up all participants' commitments gives a commitment to the total, without opening any of them.

What is sent

The commitments (a vector of curve points) are sent to the regulator with the report and stored in the audit log (feature 9). The participant does not keep r only to itself: it splits r too, like the data, and sends the shares to the three servers along with the data shares. So the servers can also aggregate r, and that is what makes the correctness check in feature 6 possible.

A signed receipt

The leader server returns a signed receipt to the participant: the report identifier, the fingerprint of the commitments, and the time of receipt. If the report is not included in the aggregation, the participant has proof that it reported on time.

Complementary industry technologies and infrastructure

- **Ristretto255 (RFC 9496):** a mathematical group over Curve25519, designed precisely for protocols like this and avoiding known pitfalls of the raw curve. Contribution: short commitments (32 bytes per cell) and fast operations.
- **curve25519-dalek (Rust):** the well-known implementation library for Ristretto255, also used in Bulletproofs implementations and Signal's protocols. Contribution: an audited, efficient implementation, without touching curve arithmetic by hand.
- **Hash-to-curve (RFC 9380):** generating the point H from a public string, so that no one knows the relation between it and G. Contribution: this is the property that makes the commitment binding. Without it, whoever chose H could cheat.

Data structure

```
CommitmentSet {
  report_id: str          # random 16 bytes, hex
  template_id: str
```

```
  commitments: list[bytes]   # L x 32 bytes, Ristretto255 points
}

Receipt {
  report_id: str
  commitments_sha256: str
  received_at: datetime
  leader_signature: bytes    # Ed25519
}
```

Input and output

Input: an encoded vector from feature 2. Output: commitments for every cell, the r values (split along with the data), and a signed receipt.

Key libraries and functions

Operation	Library	Usage
Commitment	curve25519-dalek	RistrettoPoint, Scalar, multiscalar_mul
Second generator	RFC 9380	hash_to_ristretto255("exposure-registry/H")
Receipt	ed25519-dalek	SigningKey::sign()

Feature 4: Prio3 Sharding & Validity Proofs

What does the feature do?

Splits the vector into three shares and attaches a proof that the data are valid, without revealing them. "Valid" here means: every cell is within its allowed range, and the leverage vector really is one-hot (exactly one of the five cells is 1). A partial answer to the "lying participant" threat: a participant cannot send a negative value that cancels out others, or a billion in a single cell.

How does it work?

Prio3SumVec and Prio3Histogram

The VDAF draft defines several variants of Prio3. We use two: Prio3SumVec for the exposure cells (a vector of sums, each limited to a number of bits), and Prio3Histogram for the leverage bucket: the participant supplies only the bucket number, and the proof guarantees it contributes exactly 1 to a single bucket. Each is configured by vector length and bits per cell.

How splitting with a proof works

The client breaks each value into bits and splits the bits into three random shares (as in the POC, but in a 128-bit field). Alongside, it computes a short FLP (Fully Linear Proof), which essentially says "every bit is 0 or 1, and every value is the sum of its bits". The proof is split between the servers too. Each server checks its share, the servers exchange a short check result, and if it fails, the report is discarded. No server learns anything about the values themselves.

What the proof does not prove

The proof guarantees the value is in range, not that it is true. A participant reporting 50 million when the truth is 300 million passes the check. This is the threat the threat model marked as the largest, and it has no complete cryptographic solution. Our partial answer is in feature 9: random audits in which a participant must open its commitment against its ordinary regulatory report.

Complementary industry technologies and infrastructure

- **libprio-rs:** the Rust implementation of VDAF and Prio3, maintained by ISRG (the organisation that also runs Let's Encrypt) and used in its Divvi Up service. Contribution: we use code already running in production, rather than writing FLP ourselves.
- **VDAF (draft-irtf-cfrg-vdaf):** the draft from the IETF's cryptography research group. Contribution: a public specification, with documented security proofs, that outside cryptographers can review.
- **Bulletproofs (an alternative considered):** short range proofs used in the Monero currency. Considered and rejected: they prove a range on a single commitment, but do not integrate with splitting across servers the way FLP does, and verifying them is heavier.

Data structure

```
Report {
  report_id: bytes[16]
  task_id: bytes[32]          # identifies template + period + VDAF params
  public_share: bytes
  input_shares: list[bytes]   # 3 shares, each later HPKE-encrypted (feature 8)
  commitment_randomness_shares: list[bytes]   # additive shares of r over the Ristretto255
scalar field
  token: ParticipationToken
}
VdafConfig { kind: Literal["Prio3SumVec", "Prio3Histogram"]; length: int; bits: int;
num_aggregators: 3 }
```

Input and output

Input: an encoded vector and the r values. Output: a Report with three shares and a split validity proof.

Key libraries and functions

Operation	Library	Usage
Splitting and proof	libprio-rs	Prio3::new_sum_vec(3, bits, len, chunk), shard()
Leverage bucket	libprio-rs	Prio3::new_histogram(3, 5, chunk)
Randomness	getrandom	the operating system's CSPRNG

Feature 5: Three-Aggregator Protocol

What does the feature do?

Three servers, one at each supervisory authority, receive the shares, jointly check the validity of each report, and aggregate the valid shares. At the end of the reporting period, each server holds one partial sum that looks like noise.

How does it work?

Roles

The leader server receives all reports from participants and forwards to the two helper servers their shares, encrypted so that only they can open them (feature 8). This is the structure of DAP (Distributed Aggregation Protocol), the IETF PPM working group's protocol built on VDAF. DAP is defined for two servers. We extend it to three, following the star topology the VDAF draft describes for any number of servers.

Working stages

- **Intake:** the leader checks that the token is signed by a regulator and unused (a list of spent tokens, per period), and that the task_id is valid. Duplicate reports (the same report_id) are dropped.
- **Prepare:** each server opens its share and runs Prio3's verification stage. The servers exchange short verification messages through the leader. A report that fails is marked rejected, with a reason.
- **Aggregate:** each server adds every valid report's share to its partial sum, together with the share of the r values (feature 3). Aggregation is done per batch: a group of reports from the same period.
- **Retention:** each report's shares are kept until the end of the retention period (say 90 days), to allow a binary search if a problem is found (feature 6). After that they are deleted.

Protection against reuse

Each server records which report_ids were included in which batches. A report cannot be included in two batches, which prevents anyone who gains access to a server from building an artificial "batch" containing a single report.

Complementary industry technologies and infrastructure

- **Janus:** ISRG's Rust implementation of a DAP server, which runs Divvi Up in production. Contribution: a base for our servers. It needs adapting to three servers and to our additions (the commitments).
- **PostgreSQL:** each server's database, with SERIALIZABLE isolation for aggregation operations. Contribution: even if two processes try to aggregate the same report at once, only one will succeed.
- **Kubernetes:** each server runs as a cluster, with multiple replicas and automatic recovery. Contribution: the availability answer we chose in the design decision.

Data structure

```
AggregationJob { job_id; task_id; batch_id; report_ids: list[bytes]; state:
Literal["prep", "done", "failed"] }
```

```
AggregateShare {
  task_id: bytes
  batch_id: str
  aggregator_index: int          # 0 leader, 1-2 helpers
  share: bytes                   # field-element vector
  r_share: bytes                 # sum of commitment-randomness shares
  report_count: int
  checksum: bytes[32]            # XOR of report_id hashes, for cross-check
}
```

Input and output

Input: reports from participants. Output: one AggregateShare per server per batch, and a list of rejected reports with reasons.

Key libraries and functions

Operation	Library	Usage
DAP server	Janus	aggregator, aggregation job driver
Verification	libprio-rs	prepare_init(), prepare_next(), aggregate()
Database	PostgreSQL, sqlx	SERIALIZABLE transactions

Feature 6: Verifiable Aggregate

What does the feature do?

Prio3 guarantees secrecy as long as one server is honest, but, as the draft itself says, the regulator receiving the result has to trust that the servers ran the protocol correctly. The threat model flagged this as the "server tampering with shares" threat. This feature makes the result checkable: the regulator does not need to trust; it can verify.

How does it work?

The homomorphic check

Remember the homomorphic property of the Pedersen commitment from feature 3? The regulator holds the commitments of every report in the batch (from the audit log). It adds them up and obtains a commitment to the total. When the servers submit their partial sums, the regulator adds them to get the sum X, and also the sum of their r values, R. Now it checks: does X times G plus R times H equal the sum of the commitments? If so, the sum is correct. If one server altered its share, the equality breaks.

Finding the faulty report

If the check fails, perhaps a server tampered, and perhaps a participant sent a commitment that does not match the data it split. To tell the difference, the regulator asks the servers to re-aggregate half the batch and checks it against the commitments of that half. Such a binary search finds a single bad report in about log of the number of reports steps (9 steps for 500 reports). This is why shares are retained in feature 5.

An important limitation

The check is done on the sum before privacy noise is added (feature 7). So it is available only to regulators, who are allowed to see the exact sum. The public release, with noise, cannot be checked the same way, and rests on the regulator having already checked the sum it is derived from.

Complementary industry technologies and infrastructure

- **curve25519-dalek multiscalar_mul:** computing the sum of hundreds of curve points in one operation, using Pippenger's algorithm. Contribution: checking a batch of thousands of reports in fractions of a second.
- **The "trust but verify" principle of Certificate Transparency:** just as browsers do not trust a certificate authority but check a public log, the regulator does not trust the servers but checks against commitments. Contribution: a familiar trust model, not a new one.

Data structure

```
VerificationResult {
  batch_id: str
  n_reports: int
  commitments_sum: bytes        # Ristretto point per cell
  opened_sum: list[int]
  opened_r: list[int]
  ok: bool
  bisection_trace: list[tuple[int, int, bool]] | None   # (start, end, ok)
  offending_report_id: str | None
}
```

Input and output

Input: the three AggregateShares and the commitments of every report in the batch. Output: a verified sum, or identification of the report or server that caused the failure.

Key libraries and functions

Operation	Library	Usage
Sum of commitments	curve25519-dalek	RistrettoPoint::multiscalar_mul / sum()
Binary search	Rust std	recursion over index ranges

Feature 7: Query Catalog, Threshold & Privacy Budget

What does the feature do?

Decides which questions may be asked, which cell may be opened, and how much noise to add to a release. This answers two threats from the threat model: the differencing attack (questions whose difference exposes a participant) and leakage through a result in a small cell.

How does it work?

A closed catalog

There are no free-form questions. The catalog is a list of fixed linear questions over the vector's cells, such as "total exposure at rates above 5%" (a sum of seven cells), "the share of the largest sector", or "the leverage distribution". Each question is defined as a weight vector over the cells. Adding a question to the catalog requires the approval of all three regulators and a new catalog version.

Automatic differencing check

Before a catalog version is approved, software checks every pair of questions: if the difference between them is a single cell (or group of cells) that may have fewer than K participants, the pair is rejected. This prevents the threat-model scenario ("all sectors" versus "all sectors except energy") already at design time.

The threshold gate

A cell is opened to the regulator only if the number of valid tokens that contributed to it (feature 1) is at least K=5. A blocked cell is shown as "fewer than 5 participants". A question that depends on a blocked cell returns an answer only if it includes enough open cells; otherwise it is blocked.

Noise and budget for public release

For regulators: the exact, verified sum (feature 6). For the public: the sum with Discrete Gaussian noise, calibrated to the sensitivity (the clipping cap from feature 2) and to the privacy budget. The budget is measured in zCDP (zero-Concentrated Differential Privacy), in which the budgets of several releases add up by simple addition. Each period has a total budget, every release "spends" from it, and when it runs out there are no more releases for that period. The POC showed that at epsilon 1 the question "how much exposure is above 5%" came out at 17.5% against a true 16.8%: useful even with strong privacy.

Complementary industry technologies and infrastructure

- **OpenDP:** an open differential-privacy library from Harvard and Microsoft, with a Discrete Gaussian mechanism and built-in budget accounting. Contribution: calibrated noise and budget accounting by a proven method, without implementing it ourselves.
- **zCDP (Bun and Steinke, 2016):** a privacy definition in which composing releases is simpler and tighter than under the classic definition. Contribution: more useful releases for the same budget.
- **The US Census Bureau:** used differential privacy to publish the 2020 census. Contribution: a precedent for government use of calibrated noise in official data.

Data structure

```
CatalogQuery { query_id: str; version: int; title: str; weights: dict[int, float]; public:
bool; rho_cost: float }
Catalog { version: int; queries: list[CatalogQuery]; approved_by: list[str];
differencing_check_passed: bool }
BudgetLedger { period_id: str; rho_total: float; rho_spent: float; entries:
list[tuple[str, float, datetime]] }
```

Input and output

Input: a verified sum, participant counts per cell, and the catalog. Output: answers for the regulator (exact, with blocked cells marked), answers for the public (with noise), and a budget update.

Key libraries and functions

Operation	Library	Usage
Noise	OpenDP	make_gaussian(..., mechanism discrete)
Budget	OpenDP	zCDP accounting, make_zCDP_to_approxDP
Differencing check	Python	every pair of questions, symmetric difference

Feature 8: Transport, HPKE & Operations

What does the feature do?

Ensures that every share reaches only its own server, that no one along the way can read or change it, and that the system keeps working when one server is unavailable. This answers the "one server goes down" threat within the decision we made (all three shares needed).

How does it work?

Encryption per server

The client encrypts each share with the public key of the server it is meant for, using HPKE (Hybrid Public Key Encryption, RFC 9180), as in DAP. This means that even the leader, which forwards the helpers' shares, cannot read them. It effectively holds three sealed envelopes and can open only its own. Each server's HPKE keys are published at a fixed address with a key identifier, and rotate every quarter.

Channels between servers

The servers identify each other with X.509 certificates over mTLS. Every request between servers is also signed at the message level, so the audit log can prove who sent what.

Availability

- A report that arrives while a helper is unavailable is held by the leader, still encrypted (the leader cannot open the helper's share anyway), and forwarded when the helper returns.
- A batch closes only when all three servers have finished. A delayed report is marked as such, with a reason.
- Each server runs in at least three replicas, with a replicated database.
- **Target:** 99.9% monthly availability per server. Even so, with small probability a report will be delayed precisely at a moment of stress. That is the trade-off we chose, and we wrote it down.

Keys in a secure device

Each server's private HPKE key, and the regulator's token-signing key, are kept in a hardware security module (HSM) or an equivalent cloud service, and never leave it.

Complementary industry technologies and infrastructure

- **HPKE (RFC 9180) with the hpke-rs implementation:** the standard also used in TLS Encrypted Client Hello and MLS. Contribution: modern, proven encryption in a single construction.
- **mTLS with Envoy:** mutual authentication between servers, as in any serious microservice system.
- **HSM or Cloud KMS:** keys that cannot be stolen even in a complete server compromise.
- **Prometheus and Grafana:** monitoring availability, report rate, and the share of rejected reports on each server.

Data structure

```
HpkeConfig { config_id: int; kem: "X25519-HKDF-SHA256"; kdf: "HKDF-SHA256"; aead: "AES-128-GCM"; public_key: bytes; valid_until: date }
EncryptedShare { aggregator_index: int; config_id: int; enc: bytes; ciphertext: bytes }
PendingForward { report_id; helper_index: int; encrypted_share: EncryptedShare; queued_at: datetime }
```

Input and output

Input: split shares from feature 4. Output: encrypted envelopes per server, authenticated channels, and a forwarding queue that survives failures.

Key libraries and functions

Operation	Library	Usage
Encryption	hpke-rs	setup_sender(), seal()
Channels	rustls, Envoy	mTLS
Monitoring	Prometheus	counters, histograms

Feature 9: Audit & Transparency Log

What does the feature do?

Records every significant action in a log that cannot be changed or deleted: which batches were aggregated, which questions were asked, how much privacy budget was spent, which catalog versions were approved. And it enables random audits of participants, the only partial answer to the "lying participant" threat.

How does it work?

A Merkle-tree log

Every record is added as a leaf in a Merkle tree. The tree's root is published daily. Anyone holding an old root can obtain a "consistency proof" that the new log extends it, with nothing

deleted or changed. This is exactly the mechanism of Certificate Transparency, which makes it impossible to issue a security certificate for a website in secret.

What goes into the log

- The commitments of every report (feature 3), by report_id, without identity.
- **Every batch:** identifier, number of reports, number rejected, verification result (feature 6).
- **Every question and release:** who asked, which catalog question, how much budget was spent.
- Every catalog and template version, with its approvals.

Random audits of participants

Once per period, the regulator selects a few participants at random (based on the outcome of a public, verifiable draw). A selected participant must open its commitments, that is, reveal the values and r to its own regulator only, and show that they match its ordinary regulatory report (in Europe, the Annex IV report under the AIFMD directive; in America, Form PF). A material gap leads to an inquiry. The regulator already sees the ordinary report anyway, so the audit reveals no new information. It only confirms that the encrypted report matches it.

What remains open

A random audit deters, but does not prevent. A participant who also lies in its ordinary report will not be caught here. The system cannot be more reliable than the regulatory reporting beneath it, and the reports it produces say so.

Complementary industry technologies and infrastructure

- **Trillian:** Google's open implementation of Merkle-based logs, on which the Certificate Transparency logs were built. Contribution: a proven log, with inclusion and consistency proofs ready to use.
- **drand:** a public network of "verifiable randomness" run by organisations such as Cloudflare and Protocol Labs, publishing a new random number every few seconds. Contribution: participants are chosen for audit by a number that no one, including the regulator, could have chosen in advance.
- **AIFMD Annex IV and Form PF:** the existing regulatory reports of funds in Europe and America. Contribution: an external anchor of truth for checking.

Data structure

```
LogEntry { index: int; kind: Literal["commitment", "batch", "query", "catalog",
"template", "audit"]; payload: dict; leaf_hash: bytes[32]; ts: datetime }
SignedTreeHead { tree_size: int; root_hash: bytes[32]; timestamp: datetime; signatures:
list[bytes] }   # one per regulator
AuditSelection { period_id; drand_round: int; drand_value: bytes; selected_report_ids:
list[str] }
```

Input and output

Input: events from every feature. Output: a public log with a signed daily root, inclusion and consistency proofs, and a random audit list.

Key libraries and functions

Operation	Library	Usage
Log	Trillian	QueueLeaf, GetInclusionProof, GetConsistencyProof
Public randomness	drand	GET /public/{round}
Root signature	ed25519-dalek	each regulator signs the root

Feature 10: Regulator Reports & Public Release

What does the feature do?

The output of the whole system: a dashboard for regulators with the exact, verified aggregate picture, and a periodic public release with noise. Also an export in a format that stress models and financial-stability tests can ingest.

How does it work?

The regulator dashboard

- **A heat map:** rate buckets against sectors, coloured by total exposure. Blocked cells (fewer than 5 participants) are clearly marked.
- **The leverage distribution:** a histogram of the five buckets.
- **Change from period to period:** for each cell, how much it changed and in which direction.
- **A verification badge:** for every number, a mark that it passed the commitment check (feature 6), with a link to the log entry.

The public release

Once a quarter, a short report answering the public catalog questions, with noise, and a margin of error next to every number. Because the noise distribution is known (Discrete Gaussian with known variance), a 95% interval can be computed exactly and shown, so the reader knows how precise the number is. The report also includes a clear statement of what the system knows and does not: the number of participants, the share of the market they represent, and that the data are as reliable as the regulatory reporting beneath them.

Export as distributions

Financial stress models need exactly what the system knows: not a single fund's data, but how leverage and exposures are distributed across the market. The export produces the aggregate histograms in a standard format (JSON to a documented schema, and Parquet) that an external model can load as a distribution instead of inventing one.

Permissions

Each regulator sees in the dashboard what the catalog allows. Permissions are managed in an organisational identity server (OIDC), and every dashboard view is logged.

Complementary industry technologies and infrastructure

- **Grafana or Apache Superset:** familiar dashboard tools that regulators probably already know. Contribution: no dashboard interface built from scratch.

- **SDMX:** the international standard for exchanging statistical data, used by the IMF, the ECB and the BIS. Contribution: the public release is available in the same format economists already use to download central-bank data.
- **Keycloak (OIDC):** identity and permission management for regulators.

Data structure

```
Release {
  release_id: str
  period_id: str
  audience: Literal["regulators", "public"]
  catalog_version: int
  values: dict[str, float]          # query_id -> value
  ci95: dict[str, tuple[float, float]] | None    # public only
  suppressed: list[str]
  verification_ref: int             # LogEntry index
  participants: int
}
DistributionExport { period_id; leverage_hist: list[float]; exposure_by_rate: list[float];
schema_version: str }
```

Input and output

Input: verified answers from features 6 and 7. Output: a regulator dashboard, a quarterly public report with margins of error, and distribution files for export.

Key libraries and functions

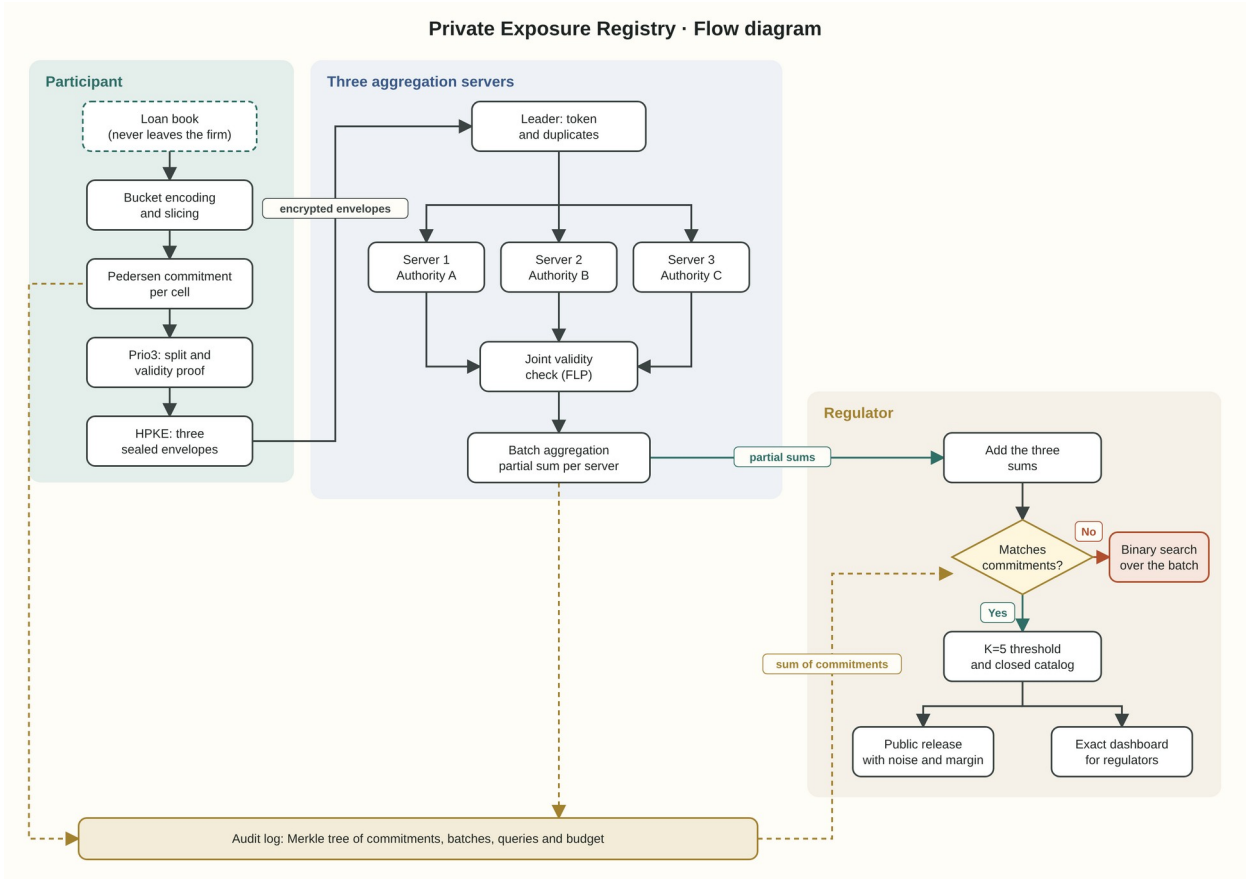
Operation	Library	Usage
Dashboards	Grafana / Superset	connected to the regulator's PostgreSQL
Release format	sdmx1 (Python)	building SDMX messages
Permissions	Keycloak	OIDC, roles

Technology table

Feature	Technologies and languages	External resources and services
1. Onboarding and identity	Privacy Pass, RSA Blind Signatures, step-ca	GLEIF API
2. Local encoding	Rust, WebAssembly, Apache Arrow	None (runs at the participant)
3. Commitments	curve25519-dalek, Ristretto255, ed25519-dalek	None
4. Splitting and proof	libprio-rs (Prio3, VDAF)	None
5. Aggregation	Janus, PostgreSQL, Kubernetes	Three servers at three authorities
6. Correctness check	curve25519-dalek	None
7. Catalog and privacy	OpenDP, Python	None
8. Transport and operations	hpke-rs, rustls, Envoy, Prometheus	HSM or Cloud KMS
9. Audit log	Trillian, ed25519-dalek	drand

Feature	Technologies and languages	External resources and services
10. Reports	Grafana / Superset, sdmx1, Keycloak	None

Flow diagram



The diagram shows three worlds that do not trust each other. At the participant (left), raw data never leave: they are encoded, locked in a commitment, and split into three encrypted envelopes, one per server. At the servers (middle), each server opens only its own envelope, the servers jointly check that the report is valid, and they aggregate. At the regulator (right), the partial sums are added, checked against the sum of commitments, and only then pass through the threshold gate and the catalog. From there two paths split: an exact dashboard for regulators, and a noised release for the public. The audit log (bottom) receives an event from every stage.

Database structure

Each party has its own database, and no single database holds "everything". Each aggregation server holds its own PostgreSQL with its own shares only. The regulator holds a PostgreSQL with participants, catalog, budget and releases. The audit log is Trillian, which uses MySQL or PostgreSQL as internal storage. The participant holds no system database at all.

On each aggregation server: table tasks

- **task_id:** BYTEA(32), primary key.

- **period_id:** VARCHAR, required.
- **template_id:** VARCHAR, required.
- **vdaf_config:** JSONB, required.
- **aggregator_index:** SMALLINT, required (0 leader, 1 and 2 helpers).

Table reports

- **report_id:** BYTEA(16), primary key.
- **task_id:** foreign key to tasks.
- **encrypted_share:** BYTEA, required (HPKE-encrypted to this server).
- **received_at:** TIMESTAMPTZ, required.
- **status:** ENUM('pending','prepared','aggregated','rejected'), required.
- **reject_reason:** VARCHAR, may be NULL.
- **batch_id:** VARCHAR, may be NULL until aggregation.
- **expires_at:** TIMESTAMPTZ, required (end of the retention period).

Table used_tokens (leader only)

- **token_hash:** BYTEA(32), primary key.
- **period_id:** VARCHAR, required.
- **used_at:** TIMESTAMPTZ, required.

Table aggregate_shares

- **task_id:** foreign key to tasks.
- **batch_id:** VARCHAR, required.
- **share:** BYTEA, required.
- **r_share:** BYTEA, required.
- **report_count:** INTEGER, required.
- **checksum:** BYTEA(32), required.
- **Composite primary key:** (task_id, batch_id).

Table pending_forwards (leader only)

- **report_id:** foreign key to reports.
- **helper_index:** SMALLINT, required.
- **encrypted_share:** BYTEA, required.
- **queued_at:** TIMESTAMPTZ, required.
- **attempts:** INTEGER, default 0.

At the regulator: table participants

- **lei:** CHAR(20), primary key.
- **regulator_id:** VARCHAR, required.
- **sector:** VARCHAR, required.
- **verified_at:** TIMESTAMPTZ, required.

- **status:** ENUM('active','suspended'), required.

Table token_issuance

- **lei:** foreign key to participants.
- **period_id:** VARCHAR, required.
- **issued_at:** TIMESTAMPTZ, required.
- **Composite primary key:** (lei, period_id). Guarantees one token per entity per period. The table does not contain the token itself, so it cannot be linked to a report.

Tables templates and catalog_versions

- **templates:** template_id (primary key), json_schema (JSONB), clip_musd, bits_per_cell, approved_at.
- **catalog_versions:** version (primary key), queries (JSONB), approved_by (an array of regulator identifiers, must contain three), differencing_check_passed (BOOLEAN, must be true).

Table batches

- **batch_id:** VARCHAR, primary key.
- **period_id:** VARCHAR, required.
- **n_reports:** INTEGER, required.
- **n_rejected:** INTEGER, required.
- **verified:** BOOLEAN, required.
- **verification_log_index:** BIGINT, required (a reference into the log).

Table budget_ledger

- **entry_id:** SERIAL, primary key.
- **period_id:** VARCHAR, required.
- **query_id:** VARCHAR, required.
- **rho:** DOUBLE PRECISION, required, positive.
- **spent_at:** TIMESTAMPTZ, required.
- **Constraint:** the sum of rho for each period_id does not exceed the defined budget (checked by a trigger before insert).

Table releases

- **release_id:** UUID, primary key.
- **period_id, audience (ENUM:** regulators, public), catalog_version, values (JSONB), ci95 (JSONB, may be NULL), suppressed (array), participants (INTEGER), verification_log_index (BIGINT).

Table audit_selections

- **period_id:** primary key.
- **drand_round:** BIGINT, required.
- **drand_value:** BYTEA, required.

- **selected_report_ids:** an array of BYTEA(16), required.
- **outcome:** JSONB (inquiry results for each selected participant), may be NULL until complete.

What the spec closed, and what remains open

- **Closed:** five of the six threats in the threat model are fully answered: Sybil (LEI and one token per entity), a tampering server (homomorphic check against commitments), denial (commitments and receipts), differencing (a closed catalog with an automatic check and a budget), and a server going down (a reasoned decision and operations). Most important: the core rests on an IETF standard in development and on code already running in production, not on cryptography we invented.
- **Open:** the sixth threat, a participant who lies. Validity proofs prove a range, not truth. Random audits deter, and strengthen the link to existing regulatory reporting, but the system is no more reliable than that reporting.
- **Open:** the system knows the state of the market right now, precisely and without exposing anyone. What it cannot do on its own is answer the question "and what happens if rates rise another point?". It shows the picture, not what will happen to it.